```
        string? line2 = await sr.ReadLineAsync(context.RequestAborted); ❹
        if (line2 is null) { return null; }                              ❺

        return double.TryParse(line1, out double height)                 ❻
            && double.TryParse(line2, out double width)                  ❻
            ? new SizeDetails(height, width)                             ❼
            : null;                                                      ❽
    }
}
```

❶ No extra attributes are needed for the SizeDetails parameter, as it has a BindAsync method.

❷ SizeDetails implements the static BindAsync method.

❸ Creates a StreamReader to read the request body

❹ Reads a line of text from the body

❺ If either line is null, indicating no content, stops processing

❻ Tries to parse the two lines as doubles

❼ If the parsing is successful, creates the SizeDetails model and returns it . . .

❽ . . . otherwise, returns null

In listing 7.8 we return `null` if parsing fails. The endpoint shown will cause the `EndpointMiddleware` to throw a `BadHttpRequestException` and return a `400` error, because the size parameter in the endpoint is required (not marked optional). You could have thrown an exception in `BindAsync`, but it wouldn't have been caught by the `EndpointMiddleware` and would have resulted in a `500` response.

7.8 Choosing a binding source

Phew! We've finally covered all the ways you can bind a request to parameters in minimal APIs. In many cases, things should work as you expect. Simple types such as `int` and `string` bind to route values and